

Ànima NLNet Application

By [Noel De Martin](#) & [REDACTED]

This document contains the same information I submitted in the [NLNet website](#) for the call of April 1st, 2026. However, it has been redacted to remove names of people who didn't want to appear in this public PDF.

Abstract

Can you explain the whole project and its expected outcome(s).

[Ànima](#) is a model-agnostic AI harness built on top of the Solid Protocol. This proposal aims to give it the ability to create Personal Applications. Aka “Vibe Coding”.

Generative AI has created a wave of new opportunities, but also came with a fair amount of issues. Many of the existing tools aren't open, and even the software they generate is problematic. With this project, we want to give everyone a Solid AI tool to create applications that are aligned with their values, and enable them to reuse data across applications.

Given the open nature of this initiative and the Solid protocol, this project will also contribute several high-impact components to the broader ecosystem, such as LLM skills and developer tools tuned for the AI era.

Background

Have you been involved with projects or organisations relevant to this project before? And if so, can you tell us a bit about your contributions?

We will have at least 2 developers working on this grant:

Noel De Martin

Noel has been involved with the Solid community since 2019. He's made some of the most beloved Solid applications, and has contributed numerous developer tools through his open source work and advocacy.

Here's a summary with some of the highlights:

- Has created multiple Solid apps ([Focus](#), [Media Kraken](#), [Umai](#)).
- Has created multiple developer libraries ([Soukai](#), [Aerogel](#)).
- Has given multiple [talks at conferences](#) (FOSDEM, Solid Symposium, Solid World).
- Has participated in the Solid Data Modules project (NLNet-funded).

[REDACTED]

[The background and description of the 2nd developer has been redacted]

[REDACTED]

[The background and description of a mentor/partner that was going to be involved in the project without drawing funds has been redacted]

Costs and Sustainability

Explain what the requested budget will be used for? Does the project have other funding sources, both past and present?

Requested Amount: 50.000€

This project doesn't currently have any funding, and the budget will be allocated to the developers building the product.

Here's a rough breakdown of the tasks:

- **LLM skills and agentic capabilities** – 320h / 16,000 €
This is the main task of the project, which is to create an LLM capable of creating Solid applications. This won't be limited to "prompt engineering", we'll need to build a lot of tooling as well. Many of the assets created here will also be reusable across the entire Solid ecosystem, since Ànima is model-agnostic. For example, this could be used to improve the existing developer tools, contribute to [the official Solid collection of LLM skills](#), or to release stand-alone Solid MCPs.
- **Schema editor** – 160h / 8,000 €
This is separated from the whole agentic workflow intentionally, in order to reduce hallucinations and show what the application is doing (for those who want to look). This could also be reused outside of this project as a visual editor for any Solid application. One important aspect will be to integrate mechanisms to find ontologies that allow FAIR data reuse, which could also be released as a stand-alone MCP. We'll build this on top of the Solid Data Modules project (NLNet-funded) and the [official Solid shapes catalogue](#).
- **User interface** – 160h / 8,000 €
Even though the brunt of the work will be handled by the models, we still need a friendly and usable UI. We want this to be usable by everyone, not just developers or people who're already familiar with product development.

- **Code management and development environment** – 120h / 6,000 €
We'll configure a development environment similar to online code sandboxes, and integrate it with git so that people can modify the code outside of the Vibe coding environment.
- **Application hosting infrastructure** – 120h / 6,000 €
We'll also need to come up with a way to serve the code. Given that applications will be PWAs, they can be hosted as static assets. But we'll also take care of setting up domains (or integrate with free domain providers) for people who aren't technical enough to host it on their own.
- **Distribution and integration with existing tooling** – 120h / 6,000 €
One of the key use-cases for this project is to use local models. There are already popular local model providers such as Ollama and LMStudio that can be used to power the code generation; this project would need to integrate with those (and others). Besides that, we'll need to configure different distribution mechanisms so that people can choose how to use this tool: self-hosted on a VPN, running in a homelab, or even installed in a personal computer as a desktop application. We can also provide a hosted version for people who don't want to deal with any of that.

Comparison to existing solutions

Compare your own project with existing or historical efforts.

There are currently a lot of Vibe-coding AI tools. But [as Noel wrote in a recent blog post](#), their defaults aren't great. Most of them generate software that is not portable nor reusable. In fact, some of them even come with vendor lock-in out of the box.

Another aspect that most tools miss is interoperability. You can create 10 task managers in a row, and none of them is going to be able to work with each other. We think this is a missed opportunity, because one of the most powerful aspects in software is reusability and composition. Especially if we're talking about Personal Software.

In that regard, Solid has an unfair advantage, because applications are much more likely to be interoperable. Even if they aren't, it's always possible to salvage and reuse the data generated by any application, given that users really are in control.

We're not aware of any existing solution that lives up to this vision (we'll see if our does!), but here's some projects worth mentioning:

[Interition](#). At face value, this looks exactly the same as what we're trying to build. But looking under the hood tells a different story, and serves as a good example that not everyone in the ecosystem shares the same values. First of all, it's not open source. The code is proprietary and it's difficult to know what's really going on under the curtain. But the real problem is that it generates applications that only work with Inrupt's PodSpaces. That completely misses the point of what Solid is all about, and given those issues we haven't spent a lot of time looking at this.

[Charlie](#). This is an idea that Tim Berners-Lee coined back in 2016, commonly referred to as "The AI that works for you". Inrupt is allegedly working on this, but [given their track record](#) we're very skeptical that this is going to live up to that initial essay. Inrupt has been focusing a lot on the enterprise sector, and it's been years since they released anything remotely "open". It's very unlikely that whatever they're working on is going to be something that can be used by individuals.

[Chef](#) by Convex. Whilst not a direct competitor, given that generated applications are not interoperable, we think in spirit this is very similar to what we're trying to build. First of all, it *is* open source, and we'll be able to learn a lot from it (it's even using the same stack as our project, JavaScript with Vercel's AI SDK). But the main reason we like it is that data is not something hidden behind the application UI, it comes with a schema visualization tool which is also AI-assisted. We're still thinking about how we'll solve this problem, but we really like the idea of exposing the data schema to app makers even if they're not technical. This could be one of the keys to making applications that are actually useful.

We're sure there are 1000 more AI projects that could be related to this in some way. We can't compare our solution to all of them, but we'll say that we've been on the lookout for a solution like this for a while, and haven't found it yet.

Technical challenges

What are significant technical challenges you expect to solve during the project, if any?

There are 3 main challenges to overcome for this project.

Ensuring reliable code generation

The main challenge for this project could be to actually get AI models to generate working applications. Fortunately, AI models are getting better at this every day, but an additional challenge is that we intend to make it provider agnostic. This makes things more difficult in the short term, but we think it's important to empower local models because they are more aligned with the ideals of the project.

There are two keys to achieve this: guardrails and evals. Guardrails are deterministic tools and libraries that are used to keep hallucinations in check. For example, using a testing workflow and using good development practices (formatting, linting, code reviews, etc.). Evals, on the other hand, are "unit tests" for AI applications. They ensure that a given AI application meets some baseline expectations. We'll be able to create an eval suite to see how well our code generation performs with different models.

This aspect of our project is something that the whole Solid ecosystem is struggling with. By working on this, we'll also be able to push the entire community forward by contributing key LLM skills, MCP servers, and developer tooling.

User Experience

Solid applications are notoriously difficult to understand for non-technical users. We'll have to make sure that apps generated with our tool don't make the same mistake. Fortunately, Noel has been working on making usable apps for a long time, so we'll make sure to embed all of his learnings and tools into the project.

Something else that should help us overcome this hurdle is the fact that this is intended for Personal Applications. That means that applications created with this tool should be quite simple, and don't need to cover a lot of edge cases (see Maggie Appleton's idea of [Barefoot Developers](#)).

Finally, AIs often suffer from "context rot", or trying to do too many things at once; which could lead to broken or unintuitive applications. The way we're decoupling the data schema from UI generation should produce much better results, given that most of the AI's job will be to build an interface on top of a robust data structure. If needed, we could potentially break down the project definition even further, by being able to indicate which screens or affordances applications should have.

Ultimately, we want users to be in control and be capable of deciding how the application they're generating actually works. We don't want to stay trapped in the "Chat interface" as many other AI tools do.

Application serving and hosting

This tool's job doesn't end with creating the application, it should also make it easier for people to actually run and use the apps. Fortunately, Solid's got our backs again because applications can run without a backend. That will simplify the deployment of apps to hosting static assets, and serving them as PWAs. Still, it's not completely a solved problem because we still have to see how to make those work in a self-hosting environment, or even running locally. We'll also need to tackle the challenge of actually running and building the code, though there are multiple open source projects that have already solved this problem (such as Sandpack). Still, it isn't completely trivial to come up with a solution that juggles all of these together.

Ecosystem

Describe the ecosystem of the project, and how you will engage with relevant actors and promote the outcomes?

The entire software industry is very excited about this type of initiative; but it's also saturated with noise. However, if we come closer to home, there haven't been as many successful initiatives about AI in the Solid ecosystem. At the same time, privacy and open source advocates are very skeptical of the role that AI will play in our future. This project shares their concerns, and addresses them by providing a solution that shares their values.

It also builds on top of some existing initiatives, and there are already some potential synergies we can envision. For example, the current implementation of Ànima already stores all interactions in the user's POD. That means that existing chat interfaces (such as SolidOS) can be used to interact with AI models through Ànima.

We've also mentioned the relation with Solid Data Modules. Their curated collection of data shapes can be baked right into Ànima, thus making sure that the generated applications fit right into the existing ecosystem.